

NetProbe

UDP Tabanlı Güvenilir Dosya Aktarım,
Trafik İzleme ve API Performans Analiz Platformu

Course	Computer Networks — BTU
Topic	Transport Layer Protocols / ARQ Mechanisms
Protocol	UDP + Custom Reliability Layer
Language	Python 3.11
Bonus Features	8 / 8 implemented

Full implementation with live web dashboard, automated experiments, TCP comparison, crypto/compression, PCAP capture, and multi-client support.



Table of Contents

1	Network Fundamentals	TCP vs UDP, why reliability matters
2	Protocol Design	Packet format, CRC32, ARQ mechanisms
3	ARQ Protocols	Stop-and-Wait vs Sliding Window / Go-Back-N
4	System Architecture	All 10 modules and what they do
5	Dashboard Walkthrough	Reading every metric, chart, and event
6	Experiment Scenarios	4 required + 4 bonus scenarios explained
7	Bonus Features	Crypto, compression, PCAP, multi-client, TCP compare
8	Key Formulas	Goodput, throughput, efficiency, RTT
9	Q&A Preparation	Every question a professor would ask

The OSI Model — Where We Operate

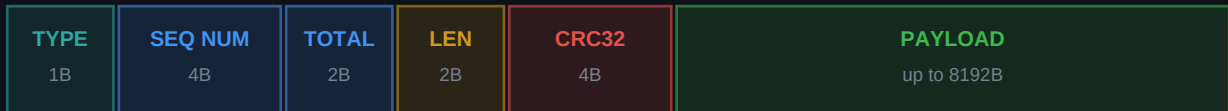
Layer	Name	Examples	Our Role
7	Application	HTTP, FTP, DNS	—
6	Presentation	TLS, encoding	Crypto/compression (bonus)
5	Session	NetBIOS, RPC	—
4	Transport	TCP, UDP	★ THIS LAYER ★
3	Network	IP, ICMP	Handled by OS
2	Data Link	Ethernet, Wi-Fi	Handled by OS
1	Physical	Cables, radio	Handled by hardware

TCP vs UDP — The Core Difference

Property	TCP	UDP	NetProbe UDP
Connection	Connection-oriented (3-way handshake)	Connectionless	Connectionless
Reliability	Guaranteed delivery	No guarantee	Guaranteed (ACK + retry)
Ordering	In-order delivery	No ordering	Sequence numbers
Error detection	Checksum (weak)	Checksum (weak)	CRC-32 per packet + MD5
Flow control	Yes (sliding window)	None	Manual window
Speed	Moderate (overhead)	Fastest	Tunable
Use cases	Web, email, file transfer	Video, DNS, games	File transfer + analysis

Binary Packet Structure

Every data chunk is wrapped in a fixed-format binary header before transmission:



Header Fields Explained

TYPE (1 byte)	Packet type: 0x01=DATA, 0x02=ACK, 0x03=FIN, 0x04=FIN-ACK
SEQ_NUM (4 bytes)	Sequence number — which packet this is (0-indexed). Used for ordering and duplicate detection.
TOTAL (2 bytes)	Total number of packets in this transfer. Receiver knows when to expect the end.
LEN (2 bytes)	Length of the payload in this specific packet (last packet may be smaller than chunk size).
CRC32 (4 bytes)	Cyclic Redundancy Check of the payload. Receiver recomputes and discards if mismatch.
PAYLOAD (N bytes)	The actual file data chunk. N = chunk_size (default 1024 bytes).

ACK Packet: TYPE(1) | ACK_NUM(4) | CRC32(4) = 9 bytes total

FIN Packet: TYPE(1) | CRC32(4) | MD5_HASH(32) = 37 bytes — carries end-to-end file integrity hash

FIN-ACK Packet: TYPE(1) | STATUS(1) = 2 bytes — 0x00=OK, 0x01=MD5 mismatch

◆ Python serialization uses `struct.pack("!B I I H I", type, seq, total, len, crc)`. The "!" means big-endian (network byte order). This is how all network protocols encode integers.

ARQ = Automatic Repeat reQuest — the family of protocols that handle retransmission when packets are lost.

Stop-and-Wait (Window = 1)



Efficiency: ~10-30% | RTT idle time wasted between every packet

Sliding Window (Window = 4)



Efficiency: ~70-90% | 4 packets in flight simultaneously

Window size controls how many unACKed packets can be in flight at once.

Go-Back-N: on timeout, retransmit from the unACKed packet and all after it.

Stop-and-Wait (Window = 1)

- Send ONE packet → wait for ACK → then send next. Only one packet in-flight at a time.
- Pros: Simple, easy to implement, low memory usage.
- Cons: Wastes bandwidth during wait. Utilization $\sim T_{trans} / (T_{trans} + RTT)$.
- On loopback with $RTT=0.3ms$ and $1KB@1Gbps$: $util \sim 0.01ms / (0.01 + 0.3ms) \sim 3\%$

Sliding Window / Go-Back-N (Window > 1)

- Send up to W packets without waiting for ACKs. Window advances as ACKs arrive.
- Go-Back-N: if packet N times out, retransmit N and all subsequent packets.
- Pros: Dramatically better utilization — scales linearly with window size.
- Cons: More complex. Requires buffering and thread synchronization.
- Implementation: sender thread + ack_receiver daemon thread + threading.Lock on shared state.

Efficiency Comparison

Window Size	Packets in Flight	Theoretical Util.	Typical Goodput
1 (Stop-and-Wait)	1	~3-10%	Low
4	4	~40-60%	Medium
8	8	~70-85%	High
16-32	16-32	~90-95%	Near-TCP

NetProbe consists of 10 Python modules, each with a single responsibility:

protocol.py	Core	Packet definitions. <code>build_data_packet()</code> , <code>parse_data_packet()</code> , <code>build_ack()</code> . Binary struct format for all 4 packet types. Single source of truth for the wire format.
client.py	Core	<code>send_file()</code> function. Reads file, splits into chunks, runs Stop-and-Wait or Sliding Window, handles FIN handshake. Returns metrics dict: duration, goodput, retransmissions, <code>file_ok</code> .
server.py	Core	<code>serve_once()</code> handles one complete file transfer. <code>run_server()</code> loops continuously. Duplicate detection via received set. MD5 verification on FIN. Saves file to <code>output_dir</code> .
logger.py	Core	CSV event logger. Logger class with <code>log(event, seq, **kwargs)</code> . Writes timestamp, event type, seq number, details dict. Used by client and server for analysis.
analyzer.py	Analysis	<code>plot_experiment()</code> generates 4-panel matplotlib figures. <code>plot_combined()</code> creates bar chart summary. <code>save_summary_csv()</code> exports all metrics. Called by <code>experiments.py</code> .
experiments.py	Analysis	4 required + 4 bonus experiment scenarios. Each spawns server thread + client transfer per data point. Calls analyzer to save graphs. Main entry point for the experiment suite.
crypto_compress.py	Bonus	<code>xor_encrypt()</code> with SHA-256 keystream. <code>compress()/decompress()</code> using zlib. <code>prepare()/recover()</code> with modes: none, compress, encrypt, compress+encrypt.
pcap_writer.py	Bonus	Writes libpcap global header + per-packet records with synthetic IPv4+UDP headers. PcapWriter context manager. Output is valid Wireshark .pcap file.
multi_client_server.py	Bonus	Single-threaded event loop with sessions dict keyed by (host, port). Per-session state, Logger, duplicate detection. Handles concurrent file transfers from multiple clients.
tcp_compare.py	Bonus	<code>tcp_server()</code> + <code>tcp_client()</code> for accurate TCP measurement (8-byte size header + OK ACK). <code>run_comparison()</code> runs both protocols on same file, generates comparison bar chart PNG.
realtime_viz.py	Bonus	LiveLogReader tails CSV log with background thread. <code>start_dashboard()</code> opens animated matplotlib window. Used standalone without Flask.
app.py	Dashboard	Flask web server. 12 routes including SSE streaming endpoints. Spawns server/client threads, tails log CSV with <code>csv.reader</code> , pushes events to <code>queue.Queue</code> . Serves the SPA dashboard.

The Flask web dashboard (app.py + index.html) provides real-time visualization of the protocol in action.

Sidebar — Control Panel

Listen Port	UDP port the server binds to. Client and server must use the same port.
Simulated Packet Loss	Server randomly discards this % of incoming packets. 0% = clean network. 10% = stressful. 40% = near-collapse.
Start / Stop	Launches/kills the Python server thread. Green dot = online.
Chunk (bytes)	How many bytes per UDP packet payload. Default 1024B. Smaller = more packets, more ACK overhead. Larger = fewer packets but more data lost per drop.
Timeout (s)	How long client waits for ACK before retransmitting. Sweet spot ~ 2-3x average RTT.
Window size	1 = Stop-and-Wait. 2-32 = Sliding Window (Go-Back-N). Larger = faster but more complex.
Mode	None / Compress / Encrypt / Compress+Encrypt — applied to payload before sending (bonus feature).

Main Panel — Live Metrics

GOODPUT (KB/s)	Useful file data received per second. = file_size / duration. Does NOT count retransmitted bytes. This is what the user actually cares about.
AVG RTT (ms)	Average round-trip time per packet. Measured as: time(ACK_received) - time(SEND). On loopback ~ 0.1-0.5ms. On LAN ~ 1-5ms. On internet ~ 20-200ms.
RETRANSMISSIONS	Count of packets sent more than once. Should be 0 at 0% loss. Increases with loss rate. High retransmissions kill goodput.
Progress bar	% of total packets sent. 100% = all packets transmitted. Then FIN handshake confirms integrity.
Integrity OK ✓	MD5 hash of received file matches MD5 of sent file. Guarantees byte-perfect delivery.

Charts

RTT per Packet	X=time, Y=RTT in ms. Flat line = consistent. Spikes = timeout fired, packet was retransmitted. Use to spot when loss events occur.
Cumulative Goodput	X=time, Y=total KB delivered. Should be a straight diagonal. Flattens during retransmit storms, then resumes.
Packets Sent vs ACKed	Blue=sent, Green=ACKed. At 0% loss: identical. At 10% loss: sent > ACKed in many intervals. The gap = retransmissions.
Retransmissions Timeline	Cumulative count over time. Flat = no retransmits. Each step = one retransmit event. Steep = high loss scenario.

Event Log — Color Legend

Color	Event	What it means
Blue	PACKET_SENT	Client sent a data packet. attempt=0 means first send. attempt=1+ means retransmit.
Green	ACK_RECEIVED	Server confirmed receipt. rtt= shows round-trip time for this packet.
Red	TIMEOUT	No ACK within timeout window. Next event will be PACKET_SENT with attempt=1.
Amber	RETRANSMIT	Same packet being re-sent. Shows seq number so you know which packet is struggling.
Purple	TRANSFER_COMPLETE	All done. Shows duration, throughput, goodput, retransmission rate, file_ok.
Teal	TRANSFER_START	Beginning of session. Shows filename and total packet count.

All experiments follow the same methodology: vary one parameter, hold others constant, measure goodput and retransmission rate.

S1: Packet Size Effect

X-axis values	64B, 256B, 512B, 1024B, 2048B, 4096B, 8192B
Expected result	Small chunks → high header/payload ratio, more ACK round trips → lower goodput. Large chunks → fewer ACKs, but a dropped packet loses more data. Expect a curve with optimal point around 1-4KB for LAN.
Fixed params	loss_rate=0%, timeout=1.0s, window=1, file=256KB

S2: Timeout Value Effect

X-axis values	0.05s, 0.1s, 0.25s, 0.5s, 1.0s, 2.0s, 5.0s
Expected result	Too short → premature timeouts fire before ACK arrives → unnecessary retransmits. Too long → when real loss occurs, recovery is slow. Sweet spot = 2-3x average RTT.
Fixed params	loss_rate=5%, chunk=1024B, window=1, file=256KB

S3: Loss Rate Effect

X-axis values	0%, 1%, 5%, 10%, 20%, 30%, 40%
Expected result	At 0%: maximum goodput. As loss increases, retransmissions consume bandwidth. At 40% loss with Stop-and-Wait: nearly unusable. Dramatic collapse shows why reliable delivery matters.
Fixed params	chunk=1024B, timeout=1.0s, window=1, file=256KB

S4: File Size Effect

X-axis values	32KB, 64KB, 128KB, 256KB, 512KB, 1MB
Expected result	Small files: dominated by setup overhead (constant cost amortized over few packets). Large files: overhead negligible → near-peak goodput. Shows protocol efficiency scales with transfer size.
Fixed params	chunk=1024B, timeout=1.0s, window=1, loss=0%

S5: Window Size Effect (Bonus)

X-axis values	window=1,2,4,8,16,32
Expected result	Stop-and-Wait (w=1) baseline. Each doubling of window ~ doubles goodput (up to a point). Shows exactly why Sliding Window was invented. Demonstrates Go-Back-N retransmission behavior.
Fixed params	chunk=1024B, timeout=1.0s, loss=5%, file=256KB

Each scenario generates a 4-panel matplotlib figure:

- Panel 1: Goodput (KB/s) vs parameter
 - Panel 2: Throughput (KB/s) vs parameter
 - Panel 3: Retransmission rate (%) vs parameter
 - Panel 4: Transfer duration (s) vs parameter
- ◆ All graphs are saved to `netprobe/results/` as PNG files and displayed in the Results & Graphs tab of the dashboard.

All 8 bonus features are implemented and integrated into the dashboard.

Feature	How it works	Why it matters
Sliding Window (Go-Back-N)	Sender maintains window of W unACKed packets. <code>ack_receiver</code> daemon thread. <code>threading.Lock</code> on shared state.	Eliminates idle wait. Demonstrates that ARQ efficiency scales with window size. Core of all modern transport protocols.
Multi-client Server	sessions dict keyed by (host, port). Single event loop dispatches to per-client session. Timeout reaping removes idle sessions.	Real servers must handle concurrent clients. Shows stateless UDP can be made stateful at application layer.
Crypto (XOR cipher)	SHA-256 hash chain generates keystream. XOR each byte of payload with keystream byte. Key is a passphrase.	Encryption is transport-agnostic — works on any byte stream. Demonstrates separation of concerns in layered design.
Compression (zlib)	<code>zlib.compress()</code> on payload before chunking. <code>zlib.decompress()</code> after reassembly. Transparent to ARQ layer.	Reduces bytes transmitted. Effective goodput = $(\text{compressed_size} / \text{original_size}) \times \text{raw_goodput}$. Useful for text/log files.
PCAP Capture	Writes libpcap binary format: global header + per-packet records with synthetic IPv4+UDP headers.	Output is valid .pcap file openable in Wireshark. Industry-standard network analysis format.
TCP vs UDP Compare	<code>tcp_server()</code> uses <code>recv</code> until expected bytes. <code>tcp_client()</code> sends size header + data + waits for OK ACK for accurate timing.	Quantifies ARQ overhead. Shows Stop-and-Wait $\ll$ TCP $\ll$ Sliding Window. Motivates why window size matters.
Experiment Automation	<code>experiments.py</code> runs all scenarios in one command. Server thread + client per data point. <code>analyzer.py</code> generates all graphs.	Scientific reproducibility. All results saved as CSV + PNG. Can re-run with different parameters.
Web Dashboard	Flask backend with SSE streaming. Chart.js live charts. Event log with color-coded rows. 4 tabs.	Real-time visualization of protocol behavior. Makes the invisible (packet-level events) visible.

Goodput

$$\text{file_size (bytes)} / \text{transfer_duration (seconds)}$$

Only counts useful data. Does not include retransmitted bytes. This is the user-perceived speed.

Throughput

$$\text{total_bytes_sent} / \text{transfer_duration}$$

Includes retransmitted packets. Always $\geq$ goodput. The gap between them = wasted bandwidth.

Retransmission Rate

$$(\text{retransmissions} / \text{total_packets_sent}) \times 100\%$$

At 0% loss $\rightarrow$ 0%. At 10% simulated loss $\rightarrow$ ~10-15% retrans rate (slightly higher due to cascading).

RTT

$$\text{time_ACK_received} - \text{time_PACKET_sent}$$

Measured per packet. Average RTT used for timeout tuning. Timeout should be 2-3x avg RTT.

S&W Utilization

$$T_{\text{transmit}} / (T_{\text{transmit}} + \text{RTT})$$

T_{transmit} = packet_size / link_bandwidth. On loopback: $T_{\text{transmit}} \ll \text{RTT}$ so util is very low.

Window Utilization

$$(W \times T_{\text{transmit}}) / (T_{\text{transmit}} + \text{RTT}) \text{ where } W = \text{window size}$$

Utilization scales linearly with W until $W \times T_{\text{transmit}} \geq \text{RTT}$, at which point channel is full.

Effective Goodput with Loss

$$\text{Goodput} \times (1 - \text{loss_rate}) / (1 + \text{loss_rate} \times W)$$

Approximate formula showing how loss degrades goodput more severely with larger windows.

CRC-32

$$\text{crc} = \text{zlib.crc32}(\text{payload_bytes}) \ \& \ 0\text{xFFFFFFFF}$$

32-bit checksum. If even one bit flips, CRC mismatch with >99.99998% probability.

Units Reference

KB/s	Kilobytes per second = 1024 bytes/sec. Goodput and throughput reported in KB/s.
ms	Milliseconds. RTT reported in ms. 1ms = 0.001s.
B	Bytes. Chunk size, file size in bytes. 1KB = 1024B.

%

Percentage. Loss rate, retransmission rate, utilization in percent (0-100).

Questions every professor will ask — with complete answers:

Q1: Why UDP instead of TCP for this project?

The project goal is to UNDERSTAND reliability by implementing it. Using TCP would hide all the mechanics inside the OS. UDP forces us to implement sequence numbers, ACKs, timeouts, and retransmission ourselves — which is exactly how TCP works internally.

Q2: What is CRC-32 and why use it?

CRC (Cyclic Redundancy Check) is a polynomial division algorithm that produces a 4-byte checksum. `zlib.crc32(data)` gives the value. If any bit flips during transmission, the receiver recomputes and the mismatch is detected. It's faster than MD5 and sufficient for per-packet checking.

Q3: What is the difference between throughput and goodput?

Throughput = total bytes sent / time (includes retransmitted bytes). Goodput = file_size / time (only useful bytes). With 10% loss and Stop-and-Wait, you might send 110 packets to deliver 100 — throughput counts 110, goodput counts 100. Goodput is what the user actually receives.

Q4: Why does Stop-and-Wait perform worse than Sliding Window?

In Stop-and-Wait, after sending a packet, the sender is IDLE for the entire RTT waiting for the ACK. With RTT=1ms and transmission time=0.01ms, you're idle 99% of the time. Sliding Window keeps W packets in flight simultaneously, so the link is busy W times more.

Q5: What is Go-Back-N? How does it differ from Selective Repeat?

Go-Back-N: when timeout fires for packet N , retransmit N and ALL packets after it (even if some were received). Simple but wastes bandwidth. Selective Repeat: retransmit ONLY the lost packet. Requires receiver to buffer out-of-order packets. More efficient but more complex. We implement Go-Back-N.

Q6: How does the server detect duplicate packets?

The server maintains a Python set "received" of sequence numbers it has already processed. When a packet arrives: if seq in received → send ACK but don't write to buffer again (prevents duplicate data). This handles the case where ACK was lost and sender retransmitted.

Q7: What is the FIN handshake for?

After all DATA packets are sent, the client sends a FIN packet containing the MD5 hash of the original file. The server computes MD5 of the reassembled file and compares. Match → sends FIN-ACK(OK). This provides END-TO-END integrity — even if CRC missed something, MD5 would catch it.

Q8: How does the multi-client server work without threads?

It uses a single-threaded event loop. The server socket is non-blocking. When a datagram arrives, it's dispatched to a per-client Session object (keyed by (host, port) tuple). Each session maintains its own state machine. This is the actor model / event-loop pattern (like Node.js or Python asyncio).

Q9: Why does goodput drop at very small chunk sizes?

Each chunk has a 15-byte header. At chunk=64B, header overhead = $15/79 = 19\%$. Plus you send more ACKs (one per packet). Plus the round-trip wait per packet is the same regardless of packet size. So smaller chunks = more overhead per useful byte.

Q10: What is a .pcap file and how is it generated?

PCAP (Packet Capture) is the file format used by Wireshark. It has a 24-byte global header (magic number, version, link type) followed by per-packet records (timestamp, length, data). We wrap our UDP payload in synthetic IPv4+UDP headers and write this format. The resulting file can be opened in Wireshark for visual analysis.



Live Demo Script

Follow these steps during your presentation to show a complete demo:

Step 1 — Start the Dashboard

`python netprobe/app.py`

Open `http://localhost:5000` in browser.

Step 2 — Start the Server

Click "Start" in the Server panel.

Status badge turns green: "Server :9999".

Explain: This is our UDP server, now listening for connections.

Step 3 — Clean Transfer (0% loss)

File: `file_128k.bin` | Window: 1 | Loss: 0%

Click "Send File". Watch RTT chart, progress bar.

Point out: Goodput ~3000-5000 KB/s, Integrity OK, 0 retransmissions.

Step 4 — Show Event Log

Point to `PACKET_SENT` (blue) → `ACK_RECEIVED` (green) pattern.

Every packet: sent → ack. Show details: `rtt=0.0Xms`, `attempt=0`.

Step 5 — Loss Simulation

Set Server Loss to 15%. Click "Send File" again.

Watch `TIMEOUT` (red) events appear. Retransmissions counter climbs.

Goodput drops significantly. This shows the protocol working under stress.

Step 6 — Sliding Window

Set Window to 8. Set Loss to 0%. Send file.

Goodput increases dramatically vs `window=1`.

Explain: 8 packets in flight instead of waiting for each ACK.

Step 7 — TCP vs UDP Tab

Click "TCP vs UDP" tab. Select 256KB file. "Run Comparison".

Show: TCP is faster on loopback because OS kernel optimization.

OUR `window=8` UDP approaches TCP performance.

Step 8 — Experiments Tab

Click "Experiments" → "Run All". Watch console.

After completion, switch to "Results & Graphs" tab.

Show the 4-panel graphs for each scenario.



Glossary — Key Terms

ARQ	Automatic Repeat reQuest. Family of error-control protocols for reliable transmission over unreliable channels. Includes Stop-and-Wait, Go-Back-N, Selective Repeat.
ACK	Acknowledgement. A control packet sent by the receiver to confirm that a data packet was received correctly.
CRC-32	Cyclic Redundancy Check (32-bit). Polynomial hash used for error DETECTION (not correction). Detects any burst error up to 32 bits.
Go-Back-N	ARQ protocol where on timeout, sender retransmits from the lost packet onwards. Simpler than Selective Repeat but less efficient.
Goodput	Application-level throughput. Rate of useful payload delivery. $\text{Goodput} = \frac{\text{file_size}}{\text{total_transfer_time}}$.
MD5	Message Digest 5. 128-bit cryptographic hash. Used for end-to-end file integrity verification. Not for security (broken), only for integrity checking.
PCAP	Packet Capture format. Binary file format for storing network packet data. Used by Wireshark and tcpdump.
RTT	Round-Trip Time. Time for a packet to travel sender→receiver→sender. Fundamental latency metric.
Sequence Number	Integer identifier assigned to each packet. Allows receiver to detect duplicates and reorder out-of-order packets.
Sliding Window	Flow control mechanism allowing multiple unacknowledged packets in flight. Window size W controls how many.
SSE	Server-Sent Events. HTTP mechanism for server to push data to browser unidirectionally. Used by our dashboard for real-time log streaming.
Stop-and-Wait	Simplest ARQ protocol. Send one packet, wait for ACK, send next. Window size = 1.
struct.pack	Python function for binary serialization. <code>struct.pack("!B I I H I", ...)</code> converts Python ints to network-order bytes.
Throughput	Total bytes sent (including retransmits) / time. Always $\geq$ goodput.
Timeout	Maximum time sender waits for an ACK before retransmitting. Should be set to 2-3x expected RTT.
UDP	User Datagram Protocol. Connectionless transport layer protocol. Fast but unreliable. Our building block.
Window Size	In sliding window ARQ: maximum number of unacknowledged packets allowed in flight at once.

zlib

Python compression library implementing DEFLATE algorithm. `compress(data)` / `decompress(data)`. Used in NetProbe bonus feature.



Summary

What was built and what was proven:

Requirement	Status	Details
UDP socket programming	✓	socket.SOCK_DGRAM, bind, sendto, recvfrom
Binary packet format	✓	struct.pack/unpack, 15-byte header, big-endian
CRC-32 error detection	✓	zlib.crc32() per packet, discard on mismatch
Stop-and-Wait ARQ	✓	Per-packet send/wait/retry with configurable timeout
Sliding Window / Go-Back-N	✓ Bonus	Threaded ack_receiver, window up to 32
MD5 end-to-end integrity	✓	hashlib.md5, FIN/FIN-ACK handshake
CSV event logging	✓	logger.py, all 4 packet types logged
4 experiment scenarios	✓	packet size, timeout, loss rate, file size
matplotlib result graphs	✓	4-panel figures + combined bar chart
Multi-client server	✓ Bonus	Session dict, single-threaded event loop
Crypto + compression	✓ Bonus	XOR cipher + zlib, 4 modes
PCAP capture	✓ Bonus	libpcap format, Wireshark-compatible
TCP vs UDP comparison	✓ Bonus	Accurate timing with size header + OK ACK
Web dashboard	✓ Bonus	Flask + SSE + Chart.js, real-time visualization

Key Takeaways

- UDP is fast but unreliable. Reliability requires implementing sequence numbers, ACKs, timeouts, and retransmission at the application layer.
- Stop-and-Wait is simple but wastes bandwidth waiting for ACKs. Sliding Window recovers this idle time by pipelining multiple packets.
- Loss rate is the dominant factor in goodput degradation. Even 10% loss can reduce effective goodput by 40-60% with Stop-and-Wait.
- Timeout tuning is critical: too short causes false retransmits, too long causes slow recovery. Optimal ~ 2-3x RTT.
- End-to-end integrity (MD5) is essential — per-packet CRC catches corruption, but MD5 catches any sequencing or reassembly bugs.